Open Access | https://doi.org/10.59350/rmayz-81x80

AI and Scientific Software: What We Learned Rebuilding Underworld3

Louis Moresi [1] [1] Australian National University

Abstract

Underworld3 has about 50,000 lines of Python/ Cython wrapping PETSc, SymPy, and a just-in-time compiler. I began a trial of AI coding tools in 2025 and they have gradually become central to the way our team works. This is a story of co-evolution as much as it is about adoption of a new set of tools.

Keywords Underworld Code, Tricks of the Trade, Auscope

Our story begins with an underworld side-project: me working on an evaluation branch of underworld3, undertaking a complicated refactor of particle / mesh interaction modules. The complexity was getting the better of me, and I wondered if it would help to have an AI tool oversee the implementation of some of the more detailed work where I was making too many mistakes.

Initially, human and AI were continually in tension. The AI tools would make rudimentary mistakes and repeat them even after re-direction. The human struggled to contain his frustration and (as a result) did not give sufficiently clear direction. The breakthrough came after a realisation that, the first thing we should have done all along was to refactor underworld3 to be AI-developer friendly. The changes that came along during that refactor made the code considerably easier for humans to learn and use as well.

1. Nothing Worked At First

AI tools initially struggled to make sense of UW3. The codebase had inconsistent patterns across modules, implicit conventions that a human could (partially) absorb over time but an AI would often miss, and it had high context requirements — you needed to understand multiple subsystems simultaneously to make changes safely.

The AI would generate plausible-looking code that was subtly wrong because it couldn't predict which pattern applied where. My first instinct was to blame the AI. But this wasn't really an AI problem, it was a code-clarity problem that AI extrapolation made worse.

2. Rewriting for AI Readability

The turning point was re-stating the goal of the exercise: **make this code work better in an AI development environment.**

We (human directing AI) removed API inconsistencies. Where one module used one access pattern and another used a different pattern for the same operation, the AI would guess wrong half the time, or even

PUBLISHED Mar 23, 2026 ARTICLE UWTN 2026-001 VERSION 1.0.0 LICENCE CC-BY-4.0 DOI 10.59350/rmayz-81x80

create a meaningless interpolation.
Fixing this for the AI fixed it for every new human contributor too.

I used the AI as a litmus test: if it consistently misused an
interface, the interface was probably confusing and I proposed changes. We evolved the API iteratively, treating the AI's mistakes as design feedback.
Predictability and consistency across modules became explicit goals.

We looked for more Pythonic approaches to the user interface — standard
patterns, familiar idioms. If a Python programmer's first instinct would be `var.data[...] = values`, then that is a better pattern to adopt than
`with mesh.access(var): var.data[...] = values`. The latter requires continual user-effort to overcome a design deficiency.

We designed developer policies around AI tool use: a machine-readable
project brief, branching conventions, commit logging styles,
planning-before-execution workflows — all so that an AI session has
enough context to be useful and enough guardrails to be safe.

Then we moved to test-driven development. Meaningful tests are
the clearest possible specification for an AI. "Make sure this test passes" is
unambiguous in a way that "fix the boundary conditions" is not.

User: The Stokes solver converges but the velocity field is symmetric top-to-bottom

when it shouldn't be. The viscosity should vary with depth. Can you

check what the solver actually sees?

AI: Let me inspect the constitutive model.

stokes.constitutive_model.flux

\boldsymbol{\sigma} = 2 \cdot 1 \cdot \dot{\boldsymbol{\epsilon}}(\mathbf{u})

The viscosity is 1 — the default value. Your depth-dependent

expression isn't reaching the solver. You defined:

viscosity_fn

\eta_0 \cdot \exp(-\gamma \cdot y)

But this was never assigned to the constitutive model.

stokes.constitutive_model.Parameters.viscosity = viscosity_fn

Now the flux is:

\boldsymbol{\sigma} = 2 \eta_0 \exp(-\gamma y) \dot{\boldsymbol{\epsilon}}(\mathbf{u})

User: Thanks

The AI doesn't search for a typo — it reads the mathematical content the solver assembled and compares it to the user's intent. The symbolic representation makes the diagnosis legible to both parties.

3. Eight Months in Four Phases

Tracking back through the git log, the work fell into four distinct phases.

3.a. Phase 1 (August–September 2025)

The first AI-assisted work tackled fundamentals: swarm and mesh variable data structures, the global evaluation routine, kdtree-based particle migration. One of the earliest commits reads "re-working structure to be more AI friendly."

The data access migration (removing `with mesh.access()` throughout the codebase in favour of direct array access) was an early success. It gave me confidence in the workflow: I was getting better at monitoring progress, steering the direction by adjusting prompts, and getting better at anticipating where and when my oversight would be most critical. I got better at specifying bounded design specifications and the appropriate tests.

3.b. Phase 2 (October–November 2025)

The new *units and non-dimensionalisation* system was a different story. This was a pervasive change touching nearly every module, and I didn't plan it well enough. I was not sure how to specify targets along the development path that we could use as stepping stones. This led to multiple false-starts, incomplete clean-up of partial implementations. Context overflowed repeatedly. There were many frustrating moments.

The AI would make changes that were locally correct but broke assumptions in modules it couldn't see. Without clear boundaries between iterations, sessions would drift, accumulating subtle inconsistencies that only showed up later.

In hindsight, this needed multiple focused passes with clear success-conditions rather than one monolithic push. But, that is easy to say, harder to do in practice when the overall architecture of the finished code was not at all clear. I was not anticipating, when we started, that adding units would change everything: symbolic compilation modules, the `PETSc` synchronisation tools, mesh-building, visualisation, and other parts of the code that had not been locked down for years.

The solution, in the end was to articulate some very clear principles that AI tools were required to explicitly remind themselves about during a session. For example: "the user must see every quantity as having units - no exceptions; if a quantity is dimensionless, that is the unit they see"; "PETSc arrays are always dimensionless"; "There is a dimensionless zone, and a zone with units; determine which side of the barrier you are on, and which gateway the data passes through".

And the clear test case was this: "here is a dimensionless version of the problem and here is an equivalent with units. The PETSc view of this problem has to be exactly the same". It is remarkable to me how long it took to state the problem in this way, and how quickly we finished the units system once we had this statement.

3.c. Phase 3 (December 2025 – January 2026)

With the API stabilising, we turned to infrastructure: a new pixi build system, PETSc 3.24 compatibility, documentation reorganisation, reinvigoraing CI/CD pipelines, Binder integration for every released version. This was steadier, more controlled work. The codebase was becoming stable enough for external users.

We started building a bank of policy documents that were human readable and AI friendly. For example: "here is how you make a release", "this is what we do when an issue arrives via GitHub", "here is how we review code and report to the underworld steering committee". We developed clear instructions for change-logs and quarterly reports against milestones. The idea of simplifying this repetitive work was enormously uplifting.

3.d. Phase 4 (February–March 2026)

We began to see significant productivity jumps. Instead of working bottom-up on the code infrastructure, we found that we could specify a target application: a
benchmark, a tutorial, a scientific problem; and develop a high-level implementation plan using the AI-assisted workflow. The code structure, the mature docstrings and documentation, and a large bank of examples (including the test suite) made this approach feasible and fast.

This has become possible because the underworld API is now consistent enough that the AI can reason about it reliably, and the test suite is robust enough to catch regressions. We learned many lessons about how to plan with AI in mind, and how to stage tasks to move cleanly up the development ladder one-working-step at a time.

4. What AI Is Good At (Underworld3, right now)

Tracing symbolic mathematics through code. Underworld3 represents its governing equations as SymPy expressions that map directly to the equations taken from textbooks and papers. This turns out to be ideal for AI-assisted development: the AI can read the mathematics, follow it through the codebase as it is transformed for numerical solution, and verify that the code implements the equations correctly. When a constitutive model or boundary condition produces wrong results, the AI can inspect the symbolic form at each stage and identify where the mathematics diverges from the intent.

Refactoring complex tasks. Changing data access patterns across
dozens of files, updating API conventions, renaming consistently. The AI reads
the whole codebase, understands the pattern, and applies it. *Caveat: any refactor should not be done in a single pass. Annotate where the code will change in case the session is interrupted. Discuss the consquences of changes. Be critical and make sure there is a path back to where you started*

Test generation and classification. Writing tests for edge cases,
classifying existing tests into reliability tiers, identifying which
tests are actually testing what they claim to test. *Tests quickly get out of date, and this is really problematic for test-driven code generation. Be very careful if AI writes the tests and also uses tests to refactor code. Only allow carefully-reviewed tests to drive coding !*

Documentation that stays current. The AI reads the implementation and produces documentation that matches what the code actually does, not what it did six months ago. *The documentation that AI likes to produce always needs review and rewriting by somebody who is familiar with the code and also familiar with the use-cases of the code. But, accurate documentation is really valuable and 100% beats placeholder comments that plague research software manuals !*

Implementing your architectural preferences. An AI tool can make your ideal software architecture a reality. You do not need to be bound by the effort involved in hand-coding complicated features. That frees you to design clearly, and with intent. "Should we do this?" is where human judgement comes in. The AI is good at "how should we do want you need?" once the decision is made.

5. Where We Are Now

Looking back over eight months, the central insight is this: making the code AI-readable has made it much easier for people to use. With consistent patterns in the user-interface, AI tools are now very capable at generating notebooks or scripts that people can use as a starting point for their own work.

We didn't try (for long) to bolt AI tools onto an existing workflow. The code changed to suit the tools, and the tools became more useful as the code improved. The result is a codebase that is better for everyone — AI and human alike.

The lesson we'd offer other scientific software teams is simple: if your AI tools are struggling with your code, listen to what that is telling you. The problem is probably real, and fixing it will pay off in ways that go well beyond AI.